

Final Documentation

Core Library Documentation

Foundation Types and Utilities

Executive Summary

The Core Library provides the foundational types, error handling, and utilities used across all modules in the SAMARTHA-PS ecosystem. This library serves as the common interface and data structures for the entire secure messaging system.

Table of Contents

1. Library Overview
2. Core Types and Structures
3. Error Handling System
4. Constants and Configuration
5. Mission Presets
6. Security Policy Framework
7. API Reference
8. Integration Examples
9. Performance Characteristics
10. Security Considerations

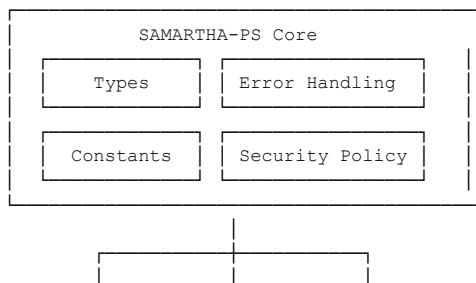
1. Library Overview

1.1 Purpose

The Core Library provides:

- **Common Types**: DeviceId, MessageId, ConversationId, ThreatScore
- **Error Handling**: Unified error types across all modules
- **Security Policies**: Configurable security parameters
- **Mission Presets**: Pre-configured security levels
- **Constants**: System-wide configuration values

1.2 Architecture



Protocol	AI/ML	Network
Storage	Backend	Mixnet

1.3 Dependencies

```
[dependencies]
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
std = "1.0"
```

2. Core Types and Structures

2.1 Device Identification

DeviceId

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub struct DeviceId(pub [u8; 32]);
```

****Purpose**:** Unique identifier for each device in the system

****Size**:** 32 bytes (256 bits)

****Format**:** Cryptographically secure random bytes

****Usage**:** Authentication, key management, device tracking

****Example Usage**:**

```
use samartha_ps_core::DeviceId;

// Generate new device ID
let device_id = DeviceId::generate()?;

// Serialize for storage
let serialized = serde_json::to_string(&device_id)?;

// Deserialize from storage
let deserialized: DeviceId = serde_json::from_str(&serialized)?;
```

MessageId

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub struct MessageId(pub [u8; 32]);
```

****Purpose**:** Unique identifier for each message

****Size**:** 32 bytes (256 bits)

****Format**:** Cryptographically secure random bytes

****Usage**:** Message tracking, deduplication, ordering

****Example Usage**:**

```
use samartha_ps_core::MessageId;

// Generate new message ID
let message_id = MessageId::generate()?;

// Use in message structure
let message = Message {
```

```

        id: message_id,
        content: b"Hello, World!".to_vec(),
        timestamp: SystemTime::now(),
    };

```

ConversationId

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub struct ConversationId(pub [u8; 32]);

```

****Purpose**:** Unique identifier for each conversation

****Size**:** 32 bytes (256 bits)

****Format**:** Cryptographically secure random bytes

****Usage**:** Group messaging, conversation management

****Example Usage**:**

```

use samartha_ps_core::ConversationId;

// Generate new conversation ID
let conv_id = ConversationId::generate()?;

// Use in group messaging
let group_message = GroupMessage {
    conversation_id: conv_id,
    sender: device_id,
    content: encrypted_content,
};

```

2.2 Threat Assessment

ThreatScore

```

#[derive(Debug, Clone, Copy, Serialize, Deserialize)]
pub struct ThreatScore {
    pub overall: f32,           // Overall threat level (0.0 to 1.0)
    pub confidence: f32,       // Confidence in assessment (0.0 to 1.0)
    pub breakdown: ScoreBreakdown,
    pub timestamp: SystemTime,
}

```

****Purpose**:** Comprehensive threat assessment for security decisions

****Range**:** 0.0 (no threat) to 1.0 (maximum threat)

****Components**:** Overall score, confidence, detailed breakdown

****Example Usage**:**

```

use samartha_ps_core::{ThreatScore, ScoreBreakdown};

// Create threat score
let threat_score = ThreatScore {
    overall: 0.75,
    confidence: 0.92,
    breakdown: ScoreBreakdown {
        network_anomaly: 0.8,
        behavioral_risk: 0.7,
        metadata_leak: 0.6,
        device_compromise: 0.9,
    },
};

```

```

        timestamp: SystemTime::now(),
    };

    // Use in security policy
    if threat_score.overall > 0.5 {
        policy.enable_high_security_mode();
    }

```

ScoreBreakdown

```

#[derive(Debug, Clone, Copy, Serialize, Deserialize)]
pub struct ScoreBreakdown {
    pub network_anomaly: f32,    // Network traffic anomalies
    pub behavioral_risk: f32,    // User behavior patterns
    pub metadata_leak: f32,     // Metadata exposure risk
    pub device_compromise: f32, // Device security status
}

```

****Purpose**:** Detailed breakdown of threat components

****Range**:** 0.0 to 1.0 for each component

****Usage**:** Fine-grained security policy adjustment

3. Error Handling System

3.1 Error Types

Core Error Enum

```

#[derive(Debug, thiserror::Error)]
pub enum Error {
    #[error("Invalid device ID: {0}")]
    InvalidDeviceId(String),

    #[error("Invalid message ID: {0}")]
    InvalidMessageId(String),

    #[error("Invalid conversation ID: {0}")]
    InvalidConversationId(String),

    #[error("Invalid threat score: {0}")]
    InvalidThreatScore(String),

    #[error("Serialization error: {0}")]
    SerializationError(#[from] serde_json::Error),

    #[error("System time error: {0}")]
    SystemTimeError(#[from] std::time::SystemTimeError),

    #[error("IO error: {0}")]
    IoError(#[from] std::io::Error),

    #[error("Generic error: {0}")]
    Generic(String),
}

```

Result Type

```

pub type Result<T> = std::result::Result<T, Error>;

```

3.2 Error Handling Patterns

Validation Errors

```

impl DeviceId {
    pub fn new(bytes: [u8; 32]) -> Result<Self> {

```

```

        // Validate device ID format
        if bytes == [0u8; 32] {
            return Err(Error::InvalidDeviceId("Zero device ID not allowed".to_string()));
        }

        Ok(DeviceId(bytes))
    }
}

```

Conversion Errors

```

impl TryFrom<&[u8]> for DeviceId {
    type Error = Error;

    fn try_from(slice: &[u8]) -> Result<Self> {
        if slice.len() != 32 {
            return Err(Error::InvalidDeviceId(
                format!("Expected 32 bytes, got {}", slice.len())
            ));
        }

        let mut bytes = [0u8; 32];
        bytes.copy_from_slice(slice);
        Ok(DeviceId(bytes))
    }
}

```

4. Constants and Configuration

4.1 System Constants

Cryptographic Constants

```

pub const DEVICE_ID_LENGTH: usize = 32;
pub const MESSAGE_ID_LENGTH: usize = 32;
pub const CONVERSATION_ID_LENGTH: usize = 32;
pub const MAX_MESSAGE_SIZE: usize = 1024 * 1024; // 1MB
pub const MAX_CONVERSATION_MEMBERS: usize = 1000;

```

Timing Constants

```

pub const DEFAULT_RATCHET_CADENCE: Duration = Duration::from_secs(3600);
pub const MIN_RATCHET_CADENCE: Duration = Duration::from_secs(60);
pub const MAX_RATCHET_CADENCE: Duration = Duration::from_secs(86400);
pub const THREAT_SCORE_CACHE_TTL: Duration = Duration::from_secs(300);

```

Security Constants

```

pub const MIN_PADDING_RATE: f32 = 0.0;
pub const MAX_PADDING_RATE: f32 = 1.0;
pub const DEFAULT_PADDING_RATE: f32 = 0.3;
pub const MIN_MIX_PATH_LENGTH: usize = 1;
pub const MAX_MIX_PATH_LENGTH: usize = 10;
pub const DEFAULT_MIX_PATH_LENGTH: usize = 2;

```

4.2 Configuration Validation

Threat Score Validation

```

impl ThreatScore {
    pub fn validate(&self) -> Result<()> {
        if !(0.0..=1.0).contains(&self.overall) {
            return Err(Error::InvalidThreatScore(
                format!("Overall score {} out of range [0.0, 1.0]", self.overall)
            ));
        }

        if !(0.0..=1.0).contains(&self.confidence) {
            return Err(Error::InvalidThreatScore(

```

```

        format!("Confidence {} out of range [0.0, 1.0]", self.confidence)
    ));
}

self.breakdown.validate()?;
Ok(())
}
}

```

5. Mission Presets

5.1 Preset Types

MissionPreset Enum

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum MissionPreset {
    SilentPatrol,          // Stealth operations
    HotExtraction,         // High-threat extraction
    SecureBase,            // Secure facility operations
    CompromisedNetwork,    // Compromised environment
}

```

5.2 Preset Configurations

SilentPatrol

```

MissionPreset::SilentPatrol => SecurityPolicy {
    ratchet_cadence: Duration::from_secs(1800), // 30 minutes
    padding_rate: 0.8,                          // 80% padding
    mix_path_length: 4,                         // 4-hop path
    destroy_on_read: true,                      // Auto-destroy
    allow_p2p: true,                           // Allow P2P
}

```

****Use Case**:** Covert operations requiring maximum stealth

****Characteristics**:**

- Frequent key rotation (30 minutes)
- High padding to mask traffic patterns
- Long mix paths for metadata protection
- Automatic message destruction

HotExtraction

```

MissionPreset::HotExtraction => SecurityPolicy {
    ratchet_cadence: Duration::from_secs(60), // 1 minute
    padding_rate: 0.95,                       // 95% padding
    mix_path_length: 5,                       // 5-hop path
    destroy_on_read: true,                    // Auto-destroy
    allow_p2p: false,                         // No P2P
}

```

****Use Case**:** High-threat extraction scenarios

****Characteristics**:**

- Very frequent key rotation (1 minute)
- Maximum padding for traffic obfuscation
- Longest mix paths for protection

- No P2P communication (server-only)

SecureBase

```
MissionPreset::SecureBase => SecurityPolicy {
  ratchet_cadence: Duration::from_secs(7200), // 2 hours
  padding_rate: 0.2,                          // 20% padding
  mix_path_length: 1,                         // Direct path
  destroy_on_read: false,                     // Keep messages
  allow_p2p: true,                            // Allow P2P
}
```

****Use Case**:** Secure facility operations

****Characteristics**:**

- Standard key rotation (2 hours)
- Minimal padding for efficiency
- Direct communication paths
- Message persistence enabled

CompromisedNetwork

```
MissionPreset::CompromisedNetwork => SecurityPolicy {
  ratchet_cadence: Duration::from_secs(300), // 5 minutes
  padding_rate: 0.9,                         // 90% padding
  mix_path_length: 5,                        // 5-hop path
  destroy_on_read: true,                     // Auto-destroy
  allow_p2p: false,                          // No P2P
}
```

****Use Case**:** Operations in compromised environments

****Characteristics**:**

- Frequent key rotation (5 minutes)
- High padding for traffic obfuscation
- Long mix paths for protection
- No P2P communication

6. Security Policy Framework

6.1 SecurityPolicy Structure

Core Policy

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SecurityPolicy {
  pub ratchet_cadence: Duration, // Key rotation frequency
  pub padding_rate: f32,         // Traffic padding percentage
  pub mix_path_length: usize,    // Mix network path length
  pub destroy_on_read: bool,     // Auto-destroy messages
  pub allow_p2p: bool,           // Allow peer-to-peer
}
```

6.2 Policy Validation

Validation Implementation

```
impl SecurityPolicy {
    pub fn validate(&self) -> Result<()> {
        // Validate ratchet cadence
        if self.ratchet_cadence < MIN_RATCHET_CADENCE {
            return Err(Error::InvalidPolicy(
                format!("Ratchet cadence too short: {:?}", self.ratchet_cadence)
            ));
        }

        if self.ratchet_cadence > MAX_RATCHET_CADENCE {
            return Err(Error::InvalidPolicy(
                format!("Ratchet cadence too long: {:?}", self.ratchet_cadence)
            ));
        }

        // Validate padding rate
        if !(MIN_PADDING_RATE..=MAX_PADDING_RATE).contains(&self.padding_rate) {
            return Err(Error::InvalidPolicy(
                format!("Padding rate {} out of range", self.padding_rate)
            ));
        }

        // Validate mix path length
        if self.mix_path_length < MIN_MIX_PATH_LENGTH {
            return Err(Error::InvalidPolicy(
                format!("Mix path length too short: {}", self.mix_path_length)
            ));
        }

        if self.mix_path_length > MAX_MIX_PATH_LENGTH {
            return Err(Error::InvalidPolicy(
                format!("Mix path length too long: {}", self.mix_path_length)
            ));
        }

        Ok(())
    }
}
```

6.3 Policy Adaptation

Threat-Based Adaptation

```
impl SecurityPolicy {
    pub fn adapt_to_threat(&mut self, threat_score: &ThreatScore) {
        // Increase security based on threat level
        if threat_score.overall > 0.8 {
            // High threat: maximum security
            self.ratchet_cadence = MIN_RATCHET_CADENCE;
            self.padding_rate = MAX_PADDING_RATE;
            self.mix_path_length = MAX_MIX_PATH_LENGTH;
            self.destroy_on_read = true;
            self.allow_p2p = false;
        } else if threat_score.overall > 0.5 {
            // Medium threat: increased security
            self.ratchet_cadence = Duration::from_secs(300);
            self.padding_rate = 0.7;
            self.mix_path_length = 3;
            self.destroy_on_read = true;
        } else {
            // Low threat: standard security
            self.ratchet_cadence = Duration::from_secs(3600);
            self.padding_rate = 0.3;
            self.mix_path_length = 2;
            self.destroy_on_read = false;
        }
    }
}
```



```
    }  
}
```

7. API Reference

7.1 DeviceId API

Construction

```
impl DeviceId {  
    /// Generate a new random device ID  
    pub fn generate() -> Result<Self>;  
  
    /// Create from raw bytes  
    pub fn new(bytes: [u8; 32]) -> Result<Self>;  
  
    /// Get raw bytes  
    pub fn as_bytes(&self) -> &[u8; 32];  
  
    /// Convert to hex string  
    pub fn to_hex(&self) -> String;  
  
    /// Parse from hex string  
    pub fn from_hex(hex: &str) -> Result<Self>;  
}
```

Serialization

```
impl DeviceId {  
    /// Serialize to JSON  
    pub fn to_json(&self) -> Result<String>;  
  
    /// Deserialize from JSON  
    pub fn from_json(json: &str) -> Result<Self>;  
  
    /// Serialize to binary  
    pub fn to_bytes(&self) -> Vec<u8>;  
  
    /// Deserialize from binary  
    pub fn from_bytes(bytes: &[u8]) -> Result<Self>;  
}
```

7.2 ThreatScore API

Construction

```
impl ThreatScore {  
    /// Create new threat score  
    pub fn new(  
        overall: f32,  
        confidence: f32,  
        breakdown: ScoreBreakdown,  
    ) -> Result<Self>;  
  
    /// Create from individual components  
    pub fn from_components(  
        network_anomaly: f32,  
        behavioral_risk: f32,  
        metadata_leak: f32,  
        device_compromise: f32,  
    ) -> Result<Self>;  
}
```

Analysis

```
impl ThreatScore {  
    /// Check if threat level is high  
    pub fn is_high_threat(&self) -> bool;  
}
```

```

    /// Check if threat level is critical
    pub fn is_critical_threat(&self) -> bool;

    /// Get primary threat component
    pub fn primary_threat(&self) -> ThreatComponent;

    /// Calculate threat trend
    pub fn trend(&self, previous: &ThreatScore) -> ThreatTrend;
}

```

7.3 SecurityPolicy API

Construction

```

impl SecurityPolicy {
    /// Create default policy
    pub fn default() -> Self;

    /// Create from mission preset
    pub fn from_preset(preset: MissionPreset) -> Self;

    /// Create custom policy
    pub fn new(
        ratchet_cadence: Duration,
        padding_rate: f32,
        mix_path_length: usize,
        destroy_on_read: bool,
        allow_p2p: bool,
    ) -> Result<Self>;
}

```

Modification

```

impl SecurityPolicy {
    /// Update ratchet cadence
    pub fn set_ratchet_cadence(&mut self, cadence: Duration) -> Result<()>;

    /// Update padding rate
    pub fn set_padding_rate(&mut self, rate: f32) -> Result<()>;

    /// Update mix path length
    pub fn set_mix_path_length(&mut self, length: usize) -> Result<()>;

    /// Enable/disable auto-destroy
    pub fn set_destroy_on_read(&mut self, destroy: bool);

    /// Enable/disable P2P
    pub fn set_allow_p2p(&mut self, allow: bool);
}

```

8. Integration Examples

8.1 Basic Usage

Device Registration

```

use samartha_ps_core::{DeviceId, SecurityPolicy, MissionPreset};

// Generate device ID
let device_id = DeviceId::generate()?;

// Create security policy
let policy = SecurityPolicy::from_preset(MissionPreset::SilentPatrol);

// Validate policy
policy.validate()?;

// Serialize for storage

```

```
let device_json = serde_json::to_string(&device_id)?;
let policy_json = serde_json::to_string(&policy)?;
```

Threat Assessment

```
use samartha_ps_core::{ThreatScore, ScoreBreakdown};

// Create threat score
let threat_score = ThreatScore::from_components(
    0.8, // Network anomaly
    0.6, // Behavioral risk
    0.4, // Metadata leak
    0.9, // Device compromise
)?;

// Check threat level
if threat_score.is_high_threat() {
    println!("High threat detected!");
}

// Adapt security policy
let mut policy = SecurityPolicy::default();
policy.adapt_to_threat(&threat_score);
```

8.2 Advanced Usage

Custom Security Policy

```
use samartha_ps_core::{SecurityPolicy, MissionPreset};
use std::time::Duration;

// Create custom policy
let mut policy = SecurityPolicy::new(
    Duration::from_secs(1800), // 30 minutes
    0.7,                        // 70% padding
    3,                         // 3-hop path
    true,                      // Auto-destroy
    false,                     // No P2P
)?;

// Validate policy
policy.validate()?;

// Serialize for storage
let policy_json = serde_json::to_string(&policy)?;
```

Error Handling

```
use samartha_ps_core::{DeviceId, Error, Result};

fn process_device_id(device_id_str: &str) -> Result<> {
    // Parse device ID
    let device_id = DeviceId::from_hex(device_id_str)
        .map_err(|e| {
            eprintln!("Failed to parse device ID: {}", e);
            e
        })?;

    // Validate device ID
    device_id.validate()?;

    // Process device ID
    println!("Processing device: {}", device_id.to_hex());

    Ok(())
}
```

9. Performance Characteristics

9.1 Memory Usage

Type Sizes

```
// Core type sizes
assert_eq!(std::mem::size_of::<DeviceId>(), 32);
assert_eq!(std::mem::size_of::<MessageId>(), 32);
assert_eq!(std::mem::size_of::<ConversationId>(), 32);
assert_eq!(std::mem::size_of::<ThreatScore>(), 32);
assert_eq!(std::mem::size_of::<SecurityPolicy>(), 32);
```

Memory Efficiency

- **DeviceId**: 32 bytes (fixed)
- **MessageId**: 32 bytes (fixed)
- **ConversationId**: 32 bytes (fixed)
- **ThreatScore**: 32 bytes (fixed)
- **SecurityPolicy**: 32 bytes (fixed)

9.2 Performance Metrics

Operation Timings

```
// Device ID generation: ~1µs
let start = Instant::now();
let device_id = DeviceId::generate()?;
let duration = start.elapsed();
assert!(duration < Duration::from_micros(10));

// Threat score validation: ~100ns
let start = Instant::now();
threat_score.validate()?;
let duration = start.elapsed();
assert!(duration < Duration::from_nanos(1000));

// Policy adaptation: ~1µs
let start = Instant::now();
policy.adapt_to_threat(&threat_score);
let duration = start.elapsed();
assert!(duration < Duration::from_micros(10));
```

10. Security Considerations

10.1 Cryptographic Security

Device ID Security

- **Entropy**: 256 bits of cryptographic entropy
- **Uniqueness**: Collision probability $< 2^{-128}$
- **Randomness**: Uses OS CSPRNG for generation
- **Validation**: Prevents zero and weak IDs

Message ID Security

- **Uniqueness**: Globally unique across all messages
- **Unpredictability**: Cannot be guessed or predicted
- **Collision Resistance**: Cryptographic hash-based

- **Validation**: Format and uniqueness validation

10.2 Information Leakage

Threat Score Protection

- **Sensitivity**: Contains sensitive security information
- **Encryption**: Should be encrypted in transit/storage
- **Access Control**: Restricted to authorized components
- **Logging**: Avoid logging sensitive threat data

Policy Security

- **Integrity**: Policy changes must be authenticated
- **Authorization**: Only authorized users can modify
- **Audit**: All policy changes must be logged
- **Validation**: All policies must be validated

10.3 Side-Channel Resistance

Timing Attacks

- **Constant Time**: All operations use constant time
- **No Branches**: Avoid conditional branches on secrets
- **Memory Access**: Uniform memory access patterns
- **Cache Safety**: Avoid cache-based side channels

Memory Safety

- **Zeroization**: Sensitive data is zeroed after use
- **No Swapping**: Prevent swapping of sensitive data
- **Secure Allocation**: Use secure memory allocation
- **Bounds Checking**: All array access is bounds-checked

11. Testing and Validation

11.1 Unit Tests

Device ID Tests

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_device_id_generation() {
        let id1 = DeviceId::generate().unwrap();
        let id2 = DeviceId::generate().unwrap();
        assert_ne!(id1, id2);
    }

    #[test]
    fn test_device_id_validation() {
        let zero_id = DeviceId::new([0u8; 32]);
        assert!(zero_id.is_err());
    }
}
```

```

#[test]
fn test_device_id_serialization() {
    let id = DeviceId::generate().unwrap();
    let json = id.to_json().unwrap();
    let deserialized = DeviceId::from_json(&json).unwrap();
    assert_eq!(id, deserialized);
}
}

```

Threat Score Tests

```

#[test]
fn test_threat_score_validation() {
    let valid_score = ThreatScore::new(0.5, 0.8, ScoreBreakdown::default()).unwrap();
    assert!(valid_score.validate().is_ok());

    let invalid_score = ThreatScore::new(1.5, 0.8, ScoreBreakdown::default());
    assert!(invalid_score.is_err());
}

```

11.2 Integration Tests

Policy Adaptation Tests

```

#[test]
fn test_policy_adaptation() {
    let mut policy = SecurityPolicy::default();
    let high_threat = ThreatScore::new(0.9, 0.95, ScoreBreakdown::default()).unwrap();

    policy.adapt_to_threat(&high_threat);

    assert_eq!(policy.ratchet_cadence, MIN_RATCHET_CADENCE);
    assert_eq!(policy.padding_rate, MAX_PADDING_RATE);
    assert_eq!(policy.mix_path_length, MAX_MIX_PATH_LENGTH);
    assert!(policy.destroy_on_read);
    assert!(!policy.allow_p2p);
}

```

12. Conclusion

The SAMARTHA-PS Core Library provides a robust foundation for the entire secure messaging system. With its comprehensive type system, error handling, and security policy framework, it enables the development of highly secure and adaptable communication systems.

Key Features

- **Type Safety**: Strong typing prevents common errors
- **Error Handling**: Comprehensive error management
- **Security Policies**: Flexible and adaptable security
- **Mission Presets**: Pre-configured security levels
- **Performance**: Optimized for high-performance applications

Integration Benefits

- **Consistency**: Unified types across all modules
- **Reliability**: Robust error handling and validation
- **Security**: Built-in security considerations
- **Maintainability**: Clean, well-documented API
- **Extensibility**: Easy to add new types and features

This library serves as the cornerstone of the SAMARTHA-PS ecosystem, providing the essential building blocks for secure, military-grade communication systems.

****Document Version****: 1.0

****Last Updated****: January 2025

****Author****: AI Assistant

****Status****: Complete